I build and run production AI automation for a live, money-critical business.

Statement pipelines, a WhatsApp assistant and finance tools in use at Trusted Figures, a London accountancy firm.

Looking for an automation, integration or solutions engineering role in London, starting February 2027.

ladopaichadze@gmail.com / LinkedIn

01 / 04

CIS statement pipeline and dashboards

- Live: intake, matching, filing and client dashboards; statement requests in first use

41	client dashboards at launch, Aug 2026[1]
335 of 351	documents received since July with a final outcome 22 Jul to 13 Oct 2026, WhatsApp and email, Oct 2026[2]

Staff see which contractor statements each client still owes, without checking by hand.

02 / 04

AI chatbot on WhatsApp and the website, with a team dashboard

- Live; one firm in production

42 of 42	acceptance benchmark cases passed live AI model, Jul 2026[3]

Clients get answers on WhatsApp and the website; staff step in from one dashboard.

03 / 04

Making Tax Digital figures from bank statements

- In internal use for quarterly Making Tax Digital submissions

140 of 140	real statements self-verified 14,801 transactions, 0 parse failures, Jul 2026[4]
9	bank statement layouts read 11 UK bank brands, 10 tested on real statements, Aug 2026[5]

Turns a sole trader's bank statements into checked figures for their quarterly tax update.

04 / 04

Business automations in n8n

- Live

13	active workflows live n8n instance, Oct 2026[6]

Client messages, documents, registrations and filing deadlines are handled in the background; staff are told what needs them.

01 of 04

CIS statement pipeline and dashboards

Staff see which contractor statements each client still owes, without checking by hand.

Status	Stack	Role
Live: intake, matching, filing and client dashboards; statement requests in first use	Python, PostgreSQL, Self-hosted OCR, OpenAI, n8n	Designed and built

41
client dashboards
at launch, Aug 2026[1]

335 of 351
documents received since July with a final outcome
22 Jul to 13 Oct 2026, WhatsApp and email, Oct 2026[2]

What it does

Clients

Send statements by WhatsApp or email, as a photo, PDF or spreadsheet.

Get a WhatsApp request in English or Georgian listing the statements still missing.

Staff

Pull each client's expected deductions from QuickBooks after one attended login.

See expected against collected for every client, down to each contractor's month.

See every held document with the plain reason it waits for a person.

Get a client-only page for each client, ready to share with its directors.

Problem

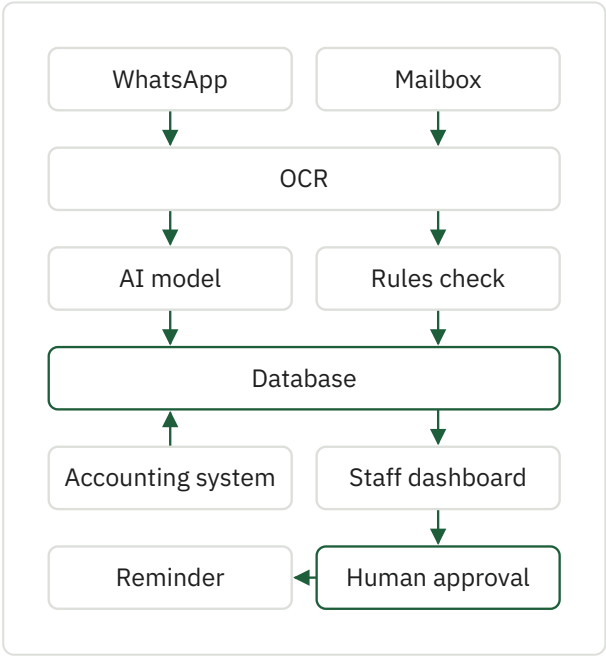
Construction subcontractors have tax deducted by every contractor they work for, and each deduction needs a statement from that contractor before it can be claimed back. Collecting them by photo and email, and tracking which are still missing, is slow work by hand.

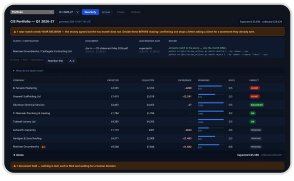
What I built

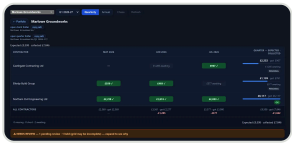
A pipeline that starts each quarter from the accounting system's list of expected deductions, then reads each statement that arrives and ticks it off against that list.

Under the hood

- Nothing is dropped: every document that arrives gets a recorded outcome, from matched to held for a person.
- Filed only when the client, contractor and month are certain; who sent it only breaks a tie. Anything unclear waits for a person.







- OCR runs on our own machine; no page image goes to a cloud OCR service. An AI model reads the extracted text and a rules check re-reads the month and amount. On a known contractor layout the rules decide; any other conflict means hold.
- Files are fingerprinted, so a re-sent statement counts once.
- Python and PostgreSQL, money as exact decimals, **3,807**[7] automated tests.

Results

- **62 of 64**[8] Q1 2026-27 statements identified were matched to what each client owed.
- **17**[9] duplicate copies caught and counted once.

Decisions and trade-offs

- Held, never guessed: client, contractor and month must be certain before a statement is filed. More documents wait for a person, but no wrong match reaches a client file.
- Statement requests are attended: a person confirms each send, one client at a time, in a step separate from composing. It costs staff time, but a wrong or repeated message cannot reach a client.

02 of 04

AI chatbot on WhatsApp and the website, with a team dashboard

Clients get answers on WhatsApp and the website; staff step in from one dashboard.

Status	Stack	Role
Live; one firm in production	TypeScript, Supabase, n8n, WhatsApp Cloud API, OpenAI, Anthropic	Designed and built

42 of 42

acceptance benchmark cases passed

live AI model, Jul 2026[3]

What it does

Clients

Ask on WhatsApp or the website chat in 6[10] languages.

Pick up later without repeating themselves: a summary and confirmed details carry over.

Register for CIS or a UTR inside the website chat, with ID photos.

Get told how to send statements when they are in this quarter's CIS collection.

Staff

Take over any thread and reply under their own name; the bot stays silent.

Send messages and files from the dashboard; outside the 24-hour window, approved templates only.

Admins

Ask about a client's Companies House deadlines on WhatsApp, in plain words.

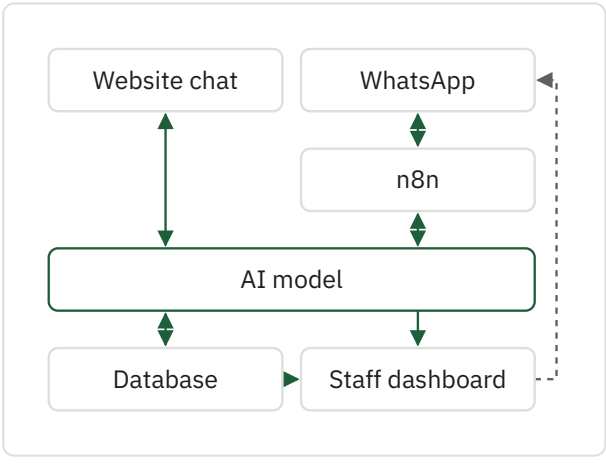
Set, confirm and cancel reminders by plain WhatsApp message.

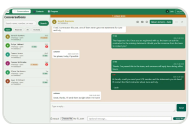
Problem

Clients and would-be clients would rather ask a question or send a photo on WhatsApp than write an email. A growing office cannot answer them all quickly and consistently without pulling staff away from the work that matters more.

What I built

An AI chatbot that answers clients on WhatsApp and on the firm's website, with a team dashboard where staff read every thread.









Under the hood

- The AI is told to answer only from the firm's service material, and code checks each reply for prices, tax rates, formatting and confidentiality before it is sent.
- The main reply call has a timeout, two retries and a circuit breaker, then fails over to a second AI provider.
- Spending caps per firm and per user, and a per-user rate limit, stop a fault or a misuse from running up AI costs.

- TypeScript on Supabase, with n8n carrying every WhatsApp message in and out; **1,349**[11] automated tests.

Decisions and trade-offs

- Redirect, never guess: tax-rate questions always get a GOV.UK link, set in code, and a client can ask for a person at any point. The bot answers less on its own, but no client gets a guessed rate.
- Unattended messages are pre-approved templates only, each checked for duplicates, opt-outs, deletion requests and business hours, and a blocked send is logged with its reason. Reminders are less flexible, but no free text reaches a client without a person.
- A reply that names a company not recorded against the person asking is replaced by a refusal. It may refuse a safe answer, and refuses in English only so far, but a reply naming another client's company does not go out.

Debugging story

A client sent eight photos at once and only some were stored. Each new file was first saved under the same placeholder name, which the database accepts only once, so photos arriving together were rejected. Each file now gets its own placeholder; a test reproduces the loss, and a live burst test after the fix stored **`8 of 8`**[12] photos, each paired to its message.

03 of 04

Making Tax Digital figures from bank statements

Turns a sole trader's bank statements into checked figures for their quarterly tax update.

Status	Stack	Role
In internal use for quarterly Making Tax Digital submissions	Python, PDF text parsing, Rules engine	Designed and built

140 of 140
real statements self-verified
14,801 transactions, 0 parse failures, Jul 2026[4]

9
bank statement layouts read
11 UK bank brands, 10 tested on real statements, Aug 2026[5]

What it does

Accountant

Load a client's PDF statements; any that fail the bank's own totals are refused.

Review every uncertain line in one list before it counts towards a figure.

Answer grouped questions on uncertain lines, largest amounts first, before saving a rule. built, not in daily use yet

Set rules for one client or a whole trade; client rules beat trade rules.

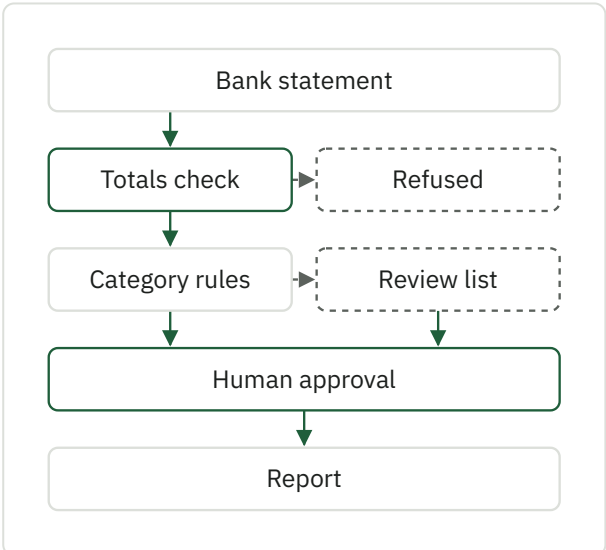
Get the SA103 boxes, a review workbook and the spreadsheet used to file.

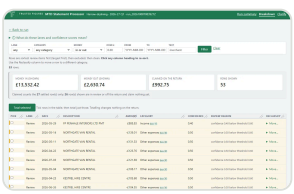
Problem

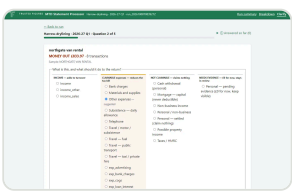
Every quarterly tax update starts from a pile of bank statements, and each line has to be read, sorted into income or expense and totalled. By hand it takes so long that a busy office turns new clients away, and one misread or missorted line changes the figure that gets filed.

What I built

A local tool that reads each bank statement and sorts every line by fixed rules. A person always reviews and files; the tool sends nothing to the tax authority.







Under the hood

- Processing is local; the tool makes no network calls. Reading and sorting use fixed rules, no AI, and a required check blocks common network and AI libraries.
- Rules sit in three layers: the client's rules beat the trade's, and the trade's beat the firm's. One client can switch off a trade rule without touching anyone else.
- Money is held as exact decimals, never floating point, and statement files are read only to the tool.

- Python, with lint, strict type checks and **976**[13] automated tests required on every change.

Results

- A rule change meant for one client would have moved a figure on another client's filed return. A before-and-after check on filed figures caught it before it landed.
- Cash taken out at a supermarket cashpoint was about to count as a claimable meal cost. An automated test caught it before any return was filed.

Decisions and trade-offs

- Fixed rules, no AI: the same statements and rules always give the same figures, and every category traces back to a rule. A new client means more questions for the accountant at first, but no figure moves without a reason.
- Read the text the PDF already holds, not a picture of the page. Scanned statements are refused for now, but every amount is taken as the bank printed it, and the totals check proves it.
- Local and single user: the page answers only on the same computer and has no login, so one person uses it today. A hosted version for more staff waits until it has proved itself on real quarters.

04 of 04

Business automations in n8n

Client messages, documents, registrations and filing deadlines are handled in the background; staff are told what needs them.

Status	Stack	Role
Live	n8n, WhatsApp Cloud API, Microsoft Graph, Companies House API, OpenAI embeddings, Supabase	Designed and built

13
active workflows
live n8n instance, Oct 2026[6]

What it does

Staff

Get WhatsApp and email reminders before accounts and confirmation statement deadlines, and when overdue.

Work from a deadline tracker that refreshes from Companies House every day.

Get a WhatsApp alert with the folder link and photo tally for each registration.

Get an alert when a filing step fails, saying what to file by hand.

Clients

Send several files at once on WhatsApp and get one combined reply.

Problem

Client work arrives on WhatsApp and through a website form, and every client company has filing deadlines to watch. By hand it is slow, and a dropped step or a missed deadline goes unnoticed until it is late.

What I built

Background workflows in n8n. The **13**[14] active workflows, by job:

Messages in

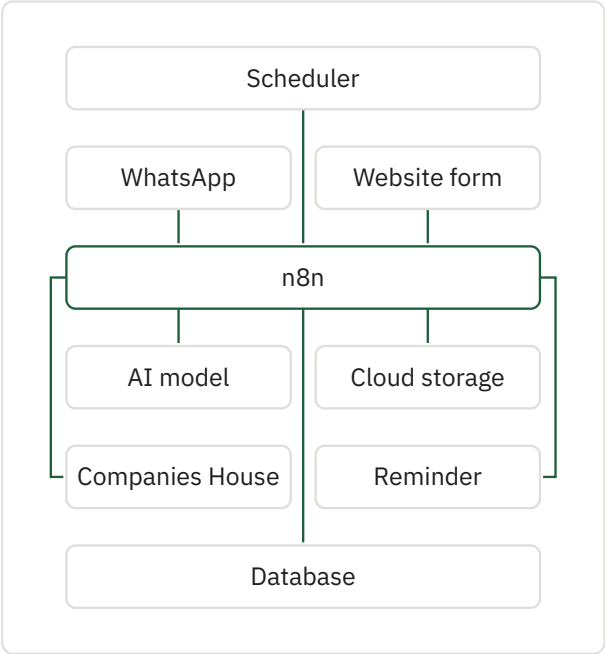
2 workflows

WhatsApp text goes to the chatbot, files to the document pipeline; an older email route runs only when started by hand.

Messages out

2 workflows

Reminders and template messages to clients leave through two guarded send workflows, never straight from an app.



Document filing

`3 workflows`

Each read document is filed in the right client folder in cloud storage; two helpers clear stale copies and empty folders.

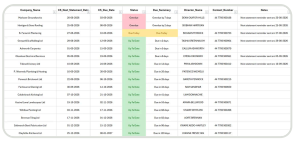
Registrations

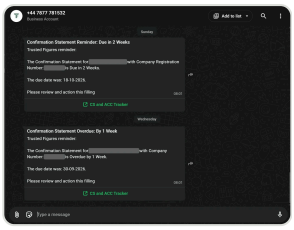
`1 workflow`

Each website registration becomes a client folder with its ID photos and a details file.

Compliance `5 workflows`

Two daily trackers, one dashboard sync and two refreshes on request.





Also built, now switched off: the chatbot's first version, which searched FAQs and HMRC guidance and drafted answers with an AI model. It never answered a client.

Under the hood

- Every call from the firm's own apps into a workflow needs a shared secret or token; no credentials sit in the workflow files.
- Reminder sends refuse duplicates with a one-time key, reserved before sending, and retry automatically.
- Repeat WhatsApp deliveries are dropped on the provider's message id.
- Incoming files are checked against allowed types and de-duplicated before they are queued.
- Only approved templates go out and blank fields are refused; filing retries before alerting staff, and a folder is deleted only when it is empty.

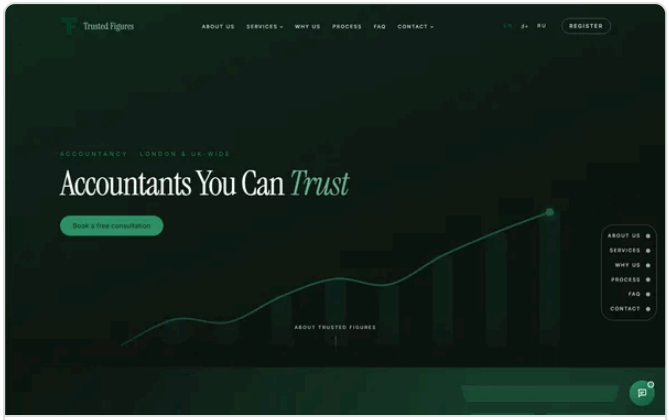
Decisions and trade-offs

- Client reminders and templates leave through two send workflows, not from each app. It adds a hop, but every client message passes the same checks.
- A failed WhatsApp reply cannot block a document from being saved: the two run separately. A client may wait for an answer, but no file is lost.
- The live workflows are exported to git, so changes can be reviewed outside n8n. The exports are only as current as the last export, but a broken edit can be compared with the last good copy.

Skills used in production

Python	01 CIS pipeline · 03 Bank statements
TypeScript	02 Chatbot
n8n	04 n8n automations · 02 Chatbot · 01 CIS pipeline
PostgreSQL / Supabase	01 CIS pipeline · 02 Chatbot · 04 n8n automations
LLM extraction with a rules cross-check	01 CIS pipeline
AI chatbots with guardrails and a fallback provider	02 Chatbot
LLM evaluation with acceptance benchmarks	02 Chatbot
Document reading: OCR and PDF text	01 CIS pipeline · 03 Bank statements
Financial data in exact decimals	01 CIS pipeline · 03 Bank statements
API integrations: WhatsApp Cloud API, Microsoft Graph, Companies House	02 Chatbot · 04 n8n automations
Browser tests with Playwright	01 CIS pipeline
CI and automated tests: GitHub Actions, pytest, strict type checks	01 CIS pipeline · 03 Bank statements · 02 Chatbot
Secret and personal data scanning	01 CIS pipeline · 02 Chatbot · 03 Bank statements
AI-assisted development with Claude Code, with an AI review of every pull request	01 CIS pipeline · 02 Chatbot · 03 Bank statements
Websites in Next.js and Tailwind, multilingual with next-intl	Websites

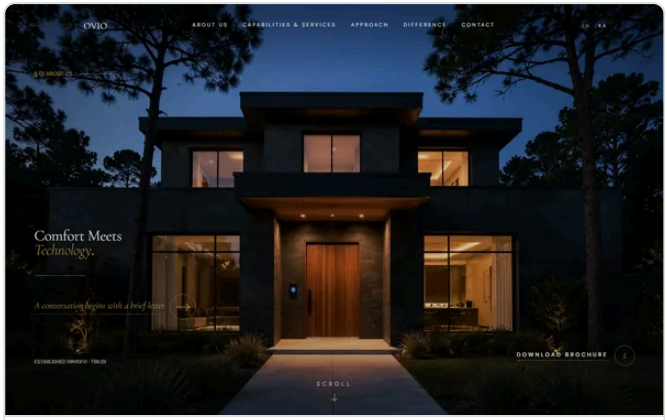
Websites



Trusted Figures

Multilingual site for a London accountancy firm, with online registration and document upload that feeds its automations.

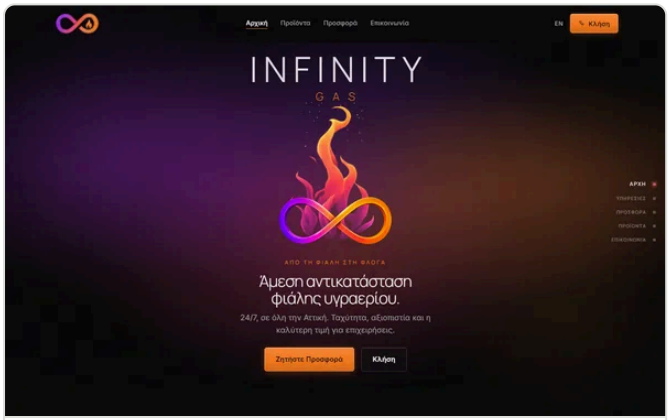
`Next.js, Supabase, Vercel`



OVIO Solutions

Bilingual Georgian and English site for a smart-home studio in Tbilisi, with a contact form that emails the studio.

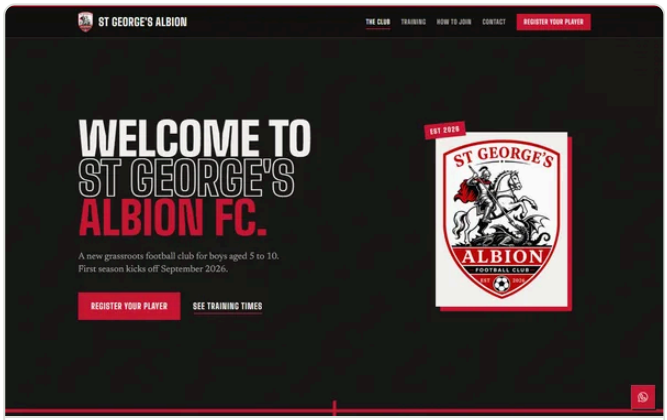
`Next.js, Tailwind, next-intl`



Infinity Gas

Greek and English site for a gas cylinder delivery business in Attica, built around a quote request form.

`Next.js, next-intl, Vitest`



St George's Albion FC

Site for a new youth football club where parents book a free session and register a player online.

`Next.js, Zod, Resend`

How I work

Every system starts with a written scope: what it does, what it must never do, and how we will know it works. Tests come before features. A person signs off every figure before it is filed and every statement request before it is sent. Each system ships with handover documents, so someone else can run it.

I build with AI coding tools. Every pull request is tested, scanned for secrets and client data, and reviewed by a second AI before it merges. What I own is the problem framing, the design and its safeguards, checking the results, and running the systems live.

Background

AI and Automation Specialist at Trusted Figures, a London accountancy firm. I started with simple scripts and n8n workflows in 2025. Production systems followed from February 2026, starting with a compliance tracker, and the WhatsApp assistant and the CIS statement pipeline from April 2026.

Languages: English, Georgian, Russian, Greek.

Contact

Write to me by email or on LinkedIn.

`ladopaichadze@gmail.com / LinkedIn`

How these figures were measured

1. 41: dashboards live on launch day, from the dated project timeline (at launch; measured Aug 2026)
2. 335 of 351: read-only count of documents received, by current outcome; final = matched, duplicate, replaced, not a CIS statement, annual statement or removed (some set by a person) (22 Jul to 13 Oct 2026, WhatsApp and email; measured Oct 2026)
3. 42 of 42: acceptance benchmark of 42 defined cases, run against the live AI model; cases passed (live AI model; measured Jul 2026)
4. 140 of 140: every statement's lines re-added to the bank's printed totals and running balance (14,801 transactions, 0 parse failures; measured Jul 2026)
5. 9: count of bank layouts the tool tries, and the brands each one names (11 UK bank brands, 10 tested on real statements; measured Aug 2026)
6. 13: workflows marked active in a read of the live instance (live n8n instance; measured Oct 2026)
7. 3,807: automated tests listed by the test runner on main (the whole CIS pipeline code; measured Oct 2026)
8. 62 of 64: read-only count of unique Q1 statements matched to an expected statement, over the Q1 statements identified; one match with its sender unconfirmed left out (Q1 2026-27, sender confirmed; measured Oct 2026)
9. 17: read-only count of documents held as duplicates: a re-sent file, or a second statement for the same month inside one PDF (since intake went live; measured Oct 2026)
10. 6: count of languages in the assistant's supported list on the main branch (the WhatsApp assistant; measured Oct 2026)
11. 1,349: automated tests run on the main branch with placeholder settings and no live services (the assistant and dashboard code; measured Oct 2026)
12. 8 of 8: photos stored and paired to their messages in a live test; two live tests before the fix stored 5 of 8 (live test after the fix; measured Sep 2026)
13. 976: automated tests listed by the test runner on main (the bank-statement processor's code; measured Oct 2026)
14. 13: active workflows grouped by job from a read of each one's steps: messages in 2, messages out 2, document filing 3, registrations 1, compliance 5 (live n8n instance, by job; measured Oct 2026)